**Aim: Installation and Setup of R/R Studio.**

**-** Install R/R Studio, explore R Studio interface, run basic commands.

Write a program to display "Welcome to R Programming".

**Solution:**

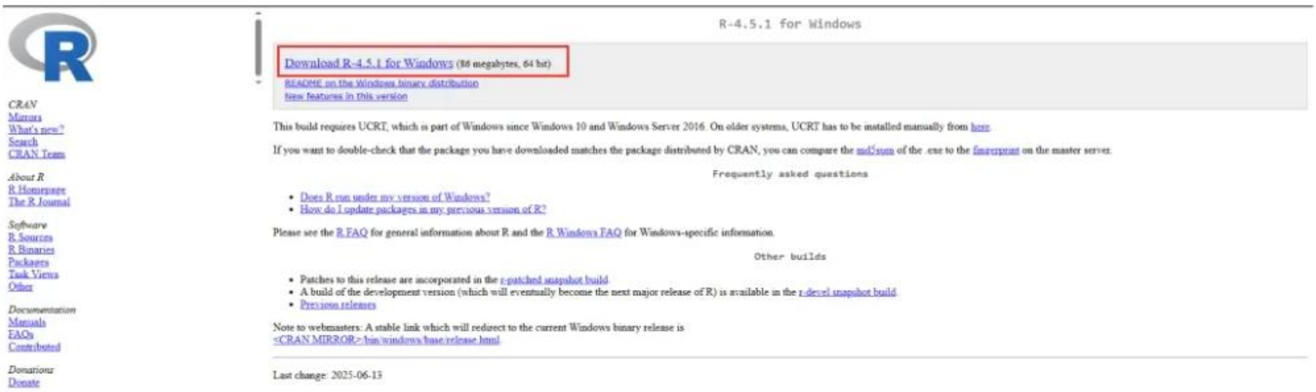
Installing R and RStudio on Windows
To Install R and R Studio on Windows we will have to download R and R Studio with the following steps.

- First, you need to set up an R environment in your local machine. Visit the official CRAN website:


- You have to download both the applications first go with R Base and then install RStudio. after click on install R you will get a new page like this.


*Download R*

- Here we can select the linux, mac or windows any one according to users system. you have to click on for which you want to install.

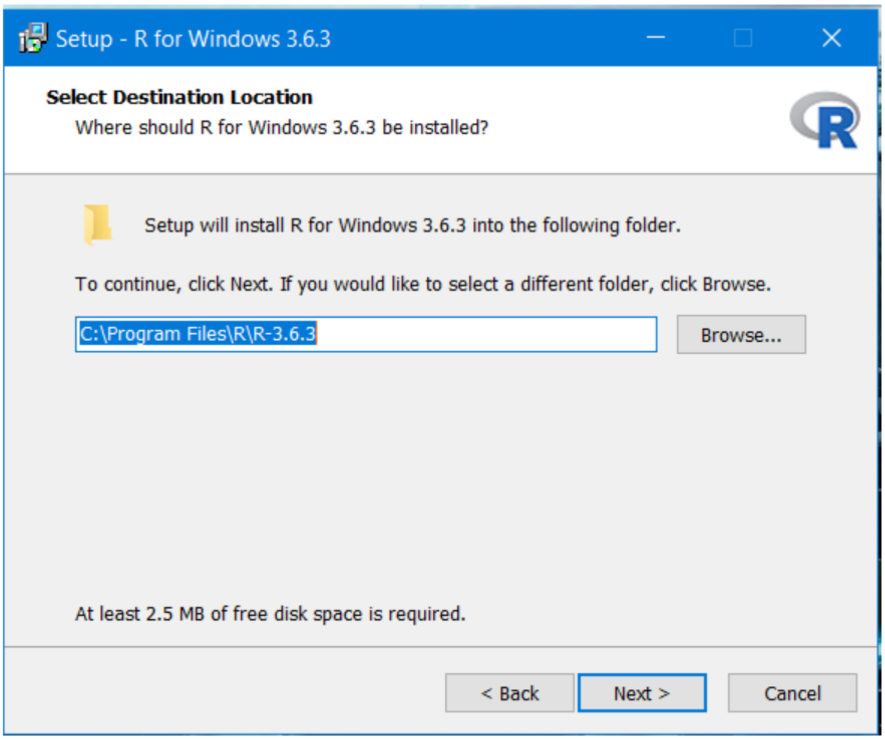



*Install R*

- now click on the link show above in image so R base start downloading and after again go to main page and download and click on Install RStudio.

## Install R and R Studio Using the Installation Wizard

**Step 1:** After downloading R for the Windows platform, install it by double-clicking it.

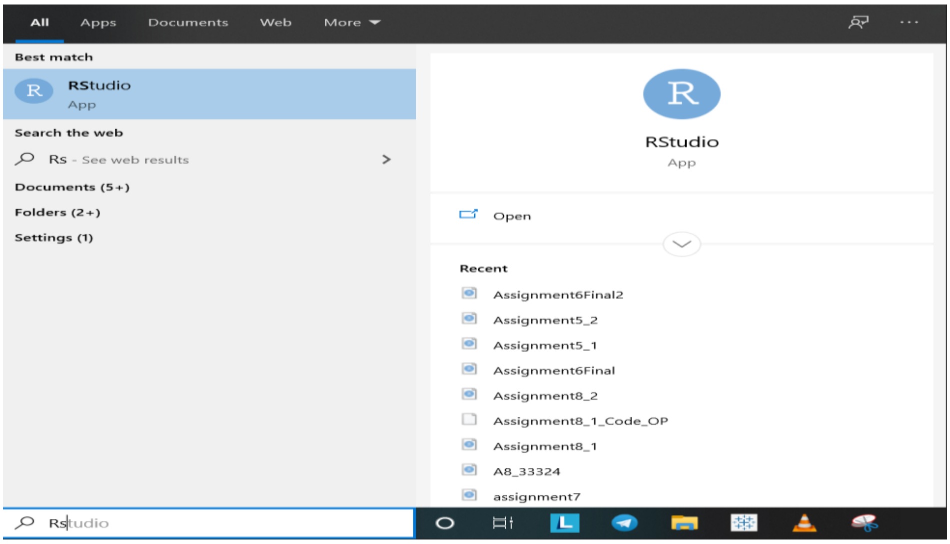



**Step 2:** Download R Studio from their official page. Note: It is free of cost (under AGPL licensing).

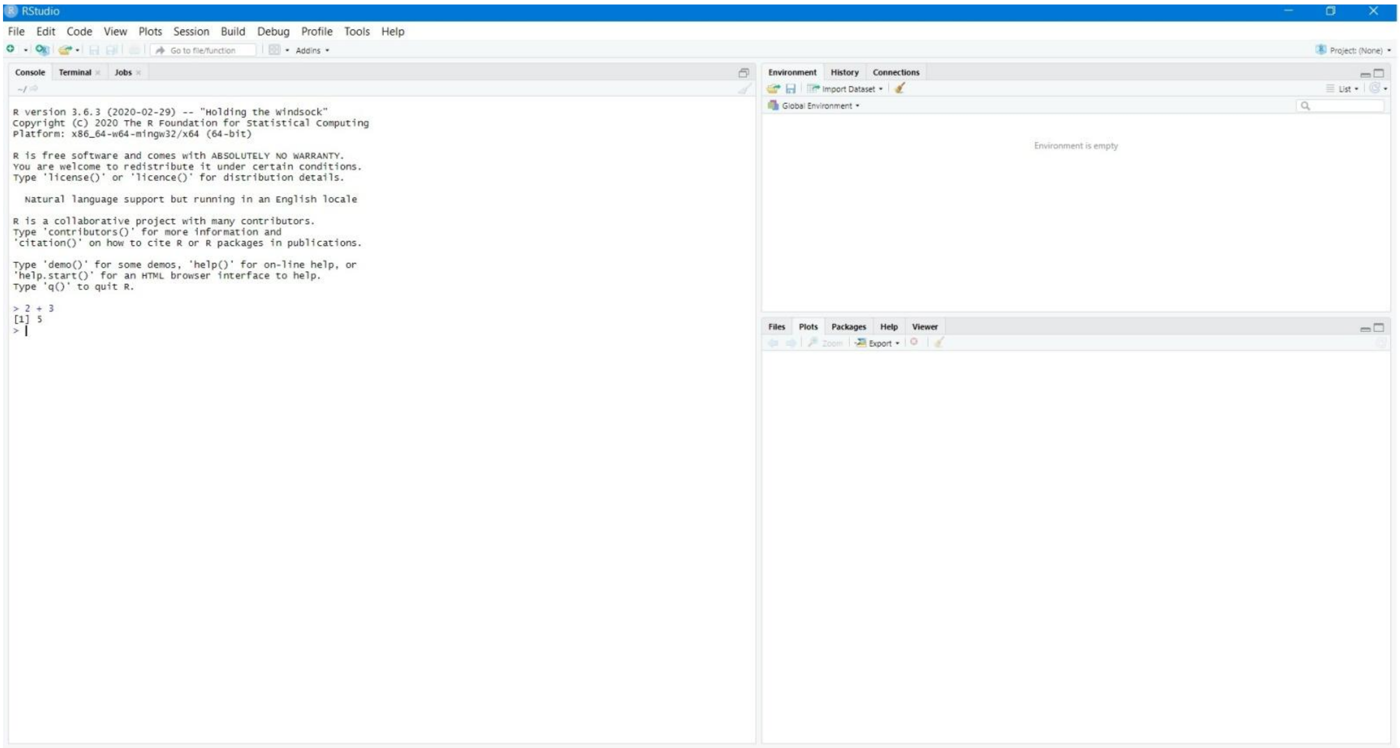
**Step 3:** After downloading, you will get a file named "RStudio-1.x.xxxx.exe" in your Downloads folder.

**Step 4:** Double-click the installer, and install the software.

**Step 5:** Search for RStudio in the Window search bar on Taskbar



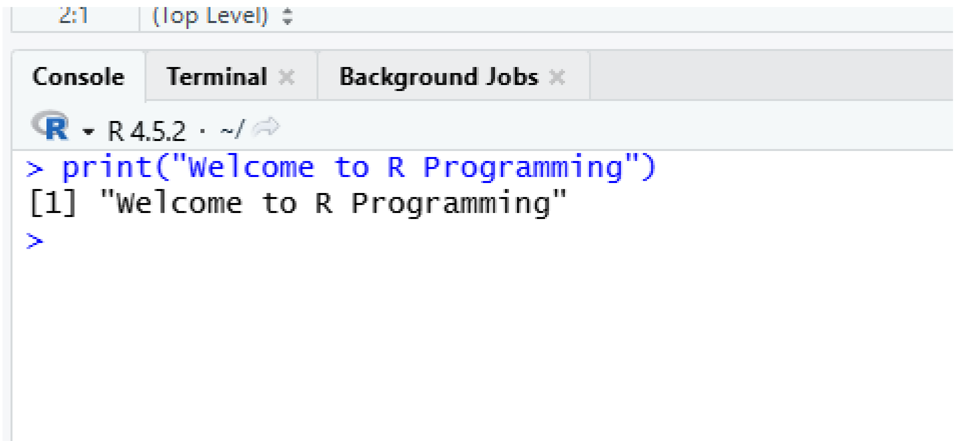
**Step 6:** Your installation is successful.




**Solution:**

print("Welcome to R Programming")

**Output:**

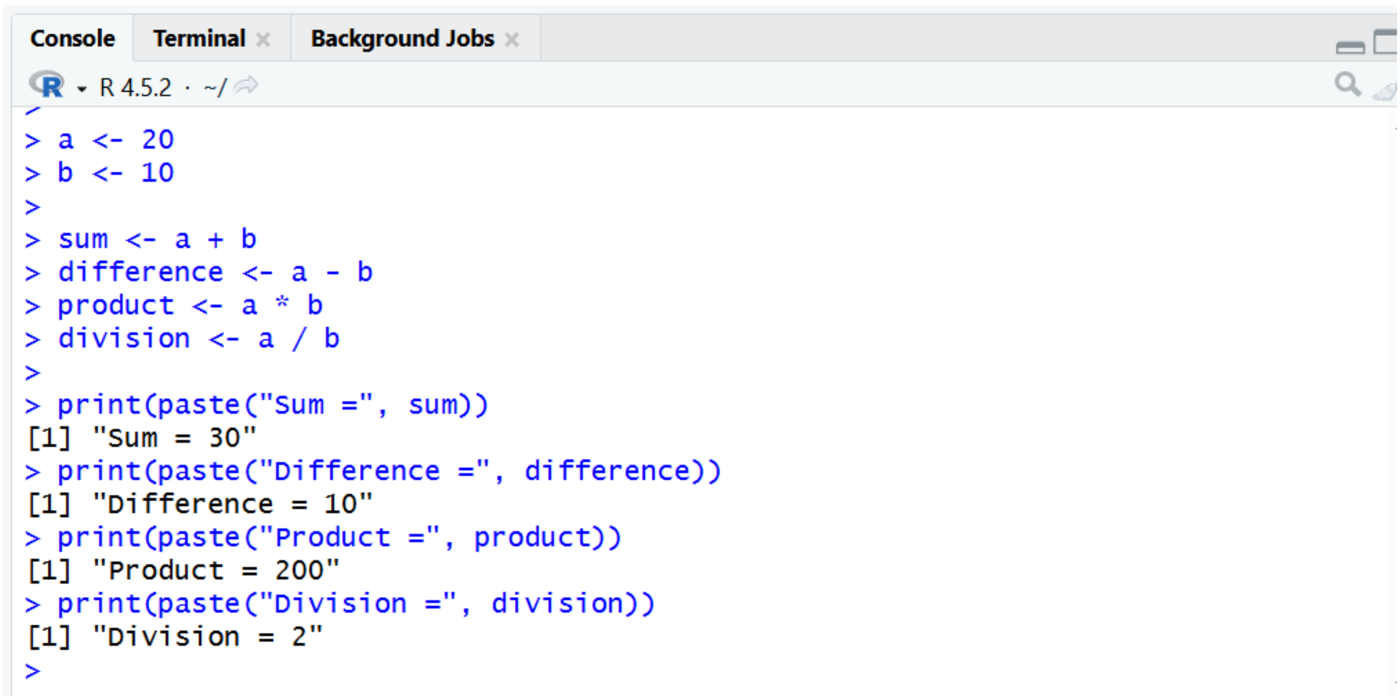


**Aim: Write an R script to calculate the sum, difference, product, and division of two numbers.**

**Solution:**

```
a <- 20
b <- 10
sum <- a + b
difference <- a - b
product <- a * b
division <- a / b
print(paste("Sum =", sum))
print(paste("Difference =", difference))
print(paste("Product =", product))
print(paste("Division =", division))
```

**Output:**



**Aim: Variables and Operators in R**

- Perform arithmetic, relational, and logical operations using variables.

**Experiment:** Create variables of different data types in R and display their type using appropriate functions

**Solution:**

```
# Arithmetic Operations
a <- 10
b <- 5
add <- a + b
sub <- a - b
mul <- a * b
div <- a / b
print(add)
print(sub)
print(mul)
print(div)
# Relational Operations
print(a > b)
print(a < b)
print(a == b)
print(a != b)
# Logical Operations
x <- TRUE
y <- FALSE
print(x & y)
print(x | y)
```

```
print(!x)

# Variables of Different Data Types

num <- 25

decimal <- 10.5

text <- "R Programming"

flag <- TRUE

print(typeof(num))

print(typeof(decimal))

print(typeof(text))

print(typeof(flag))
```

**Output:**

```
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
> # Arithmetic Operations
> a <- 10
> b <- 5
>
> add <- a + b
> sub <- a - b
> mul <- a * b
> div <- a / b
>
> print(add)
[1] 15
> print(sub)
[1] 5
> print(mul)
[1] 50
> print(div)
[1] 2
>
> # Relational Operations
> print(a > b)
[1] TRUE
> print(a < b)
[1] FALSE
> print(a == b)
[1] FALSE
> print(a != b)
[1] TRUE
>
> # Logical Operations
> x <- TRUE
> y <- FALSE
>
> print(x & y)
[1] FALSE
> print(x | y)
[1] TRUE
> print(!x)
[1] FALSE
>
> # Variables of Different Data Types
> num <- 25
> decimal <- 10.5
> text <- "R Programming"
> flag <- TRUE
>
> print(typeof(num))
[1] "double"
> print(typeof(decimal))
[1] "double"
> print(typeof(text))
[1] "character"
> print(typeof(flag))
[1] "logical"
> |
```

**Aim: Vectors and Vectorized Operations- Create vectors and apply vectorized arithmetic operations.**

**Experiment:**

1. Write a R program to perform following operations on Vector: min(), max(), mean(), sqrt(), length(), sum(), prod(), sort()(in ascending and descending order), rev(), addition/subtraction/multiplication/division of two vectors.

2. Write a R program to create a list containing a vector, a matrix and a list and perform following operations:

i. Update the elements in the list.

ii. Merge two lists into one list.

iii. Count number of objects in a given list.

**Solution:**

```
v1 <- c(2, 4, 6, 8, 10)

v2 <- c(1, 3, 5, 7, 9)

min(v1)

max(v1)

mean(v1)

sqrt(v1)

length(v1)

sum(v1)

prod(v1)

sort(v1)

sort(v1, decreasing=TRUE)

rev(v1)

v1 + v2

v1 - v2

v1 * v2
```

| | | |
|---|---|---|
| MMICT&BM | **MM INSTITUTE OF COMPUTER TECHNOLOGY & BUSINESS MANAGEMENT-MULLANA** | **LABORATORY MANUAL** |
| | **PRACTICAL EXPERIMENT INSTRUCTION SHEET** | |
| | **Title : 1.Write a R program to perform following operations on Vector: min(), max(), mean(), sqrt(), length(), sum(), prod(), sort()(in ascending and descending order), rev(), addition/subtraction/multiplication/division of two vectors.** | |

| | | |
|---|---|---|
| EXPERIMENT NO.: **BCA- 602** | ISSUE No. : 01 | ISSUE DATE: |
| REV. DATE: | REV. No.: 00 | Deptt. MCA |
| LABORATORY: R Programming | Semester-VI | Page: 10of 47 |

```
vec <- c(10, 20, 30)

mat <- matrix(1:6, nrow=2)

sublist <- list("A", "B", "C")

my_list <- list(vec, mat, sublist)

my_list[[1]][2] <- 99

my_list[[2]][1,1]  <- 50

my_list[[3]][2] <- "Z"

list2 <- list(100, 200, 300)

merged_list <- c(my_list, list2)

length(my_list)
```

**Output:**

```
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
> v1 <- c(2, 4, 6, 8, 10)
> v2 <- c(1, 3, 5, 7, 9)
> min(v1)
[1] 2
> max(v1)
[1] 10
> mean(v1)
[1] 6
> sqrt(v1)
[1] 1.414214 2.000000 2.449490 2.828427 3.162278
> length(v1)
[1] 5
> sum(v1)
[1] 30
> prod(v1)
[1] 3840
> sort(v1)
[1]  2  4  6  8 10
> sort(v1, decreasing=TRUE)
[1] 10  8  6  4  2
> rev(v1)
[1] 10  8  6  4  2
> v1 + v2
[1]  3  7 11 15 19
> v1 - v2
[1] 1 1 1 1 1
> v1 * v2
[1]  2 12 30 56 90
> v1 / v2
[1] 2.000000 1.333333 1.200000 1.142857 1.111111
> vec <- c(10, 20, 30)
> mat <- matrix(1:6, nrow=2)
> sublist <- list("A", "B", "C")
> my_list <- list(vec, mat, sublist)
> my_list[[1]][2] <- 99
> my_list[[2]][1,1] <- 50
> my_list[[3]][2] <- "Z"
> list2 <- list(100, 200, 300)
> merged_list <- c(my_list, list2)
> length(my_list)
[1] 3
> 
```

**Aim: Subsetting Data in R**

Subset vectors using indexing, logical conditions, and named elements.

**Experiment:** Create a vector of 10 elements and extract even-indexed elements.

**Solution:**

```
v <- c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50)

v[1]

v[3]

v[c(1, 3, 5)]

v[v > 25]

names(v) <- c("a","b","c","d","e","f","g","h","i","j")

v["d"]

v[seq(2, length(v), by = 2)]
```

**Output:**

```
9:1    (Top Level) ÷
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/

> v <- c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50)
> v[1]
[1] 5
> v[3]
[1] 15
> v[c(1, 3, 5)]
[1]  5 15 25
> v[v > 25]
[1] 30 35 40 45 50
> names(v) <- c("a","b","c","d","e","f","g","h","i","j")
> v["d"]
 d
20
> v[seq(2, length(v), by = 2)]
 b  d  f  h  j
10 20 30 40 50
> 
```

**Aim: Handling NA and NULL Values**

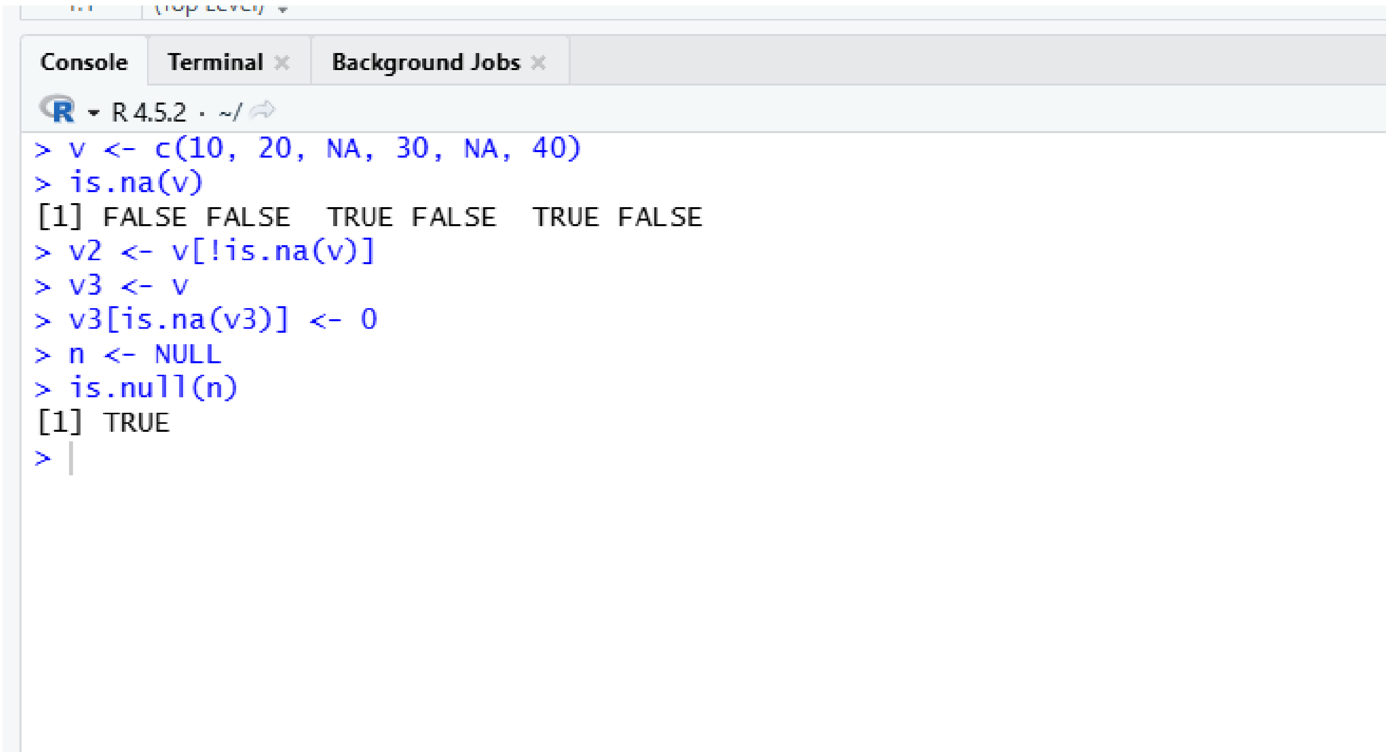
Detect, remove, and replace NA and NULL values; apply coding standards.

**Experiment:** Create a vector with NA values and write a program to remove them.

**Solution:**

```
v <- c(10, 20, NA, 30, NA, 40)

is.na(v)

v2 <- v[!is.na(v)]

v3 <- v

v3[is.na(v3)] <- 0

n <- NULL

is.null(n)
```

**Output:**



**Aim: Conditional Statements Using if if-else and Nested if-else Statements**

Implement decision-making using if statements.

**Experiment:** Write a program to check leap year in R.

**Solution:**

```
year <- 2024
if (year %% 400 == 0) {
  cat("Leap Year")
} else if (year %% 100 == 0) {
  cat("Not a Leap Year")
} else if (year %% 4 == 0) {
  cat("Leap Year")
} else {
  cat("Not a Leap Year")
}
```

**Output:**

27:1 (Top Level)

Console Terminal Background Jobs

R 4.5.2 · ~/

```
+ year <- 2024
+ if (year %% 400 == 0) {
+   cat("Leap Year")
+ } else if (year %% 100 == 0) {
+   cat("Not a Leap Year")
+ } else if (year %% 4 == 0) {
+   cat("Leap Year")
+ } else {
+   cat("Not a Leap Year")
+ }
+
+ Leap Year
```

**Aim: For Loop Implementation and While Loop Implementation**

**Use for loop for repetitive tasks and sequence generation.**

**Experiment:** 1. Write a user defined function to generate Fibonacci series using while loop.

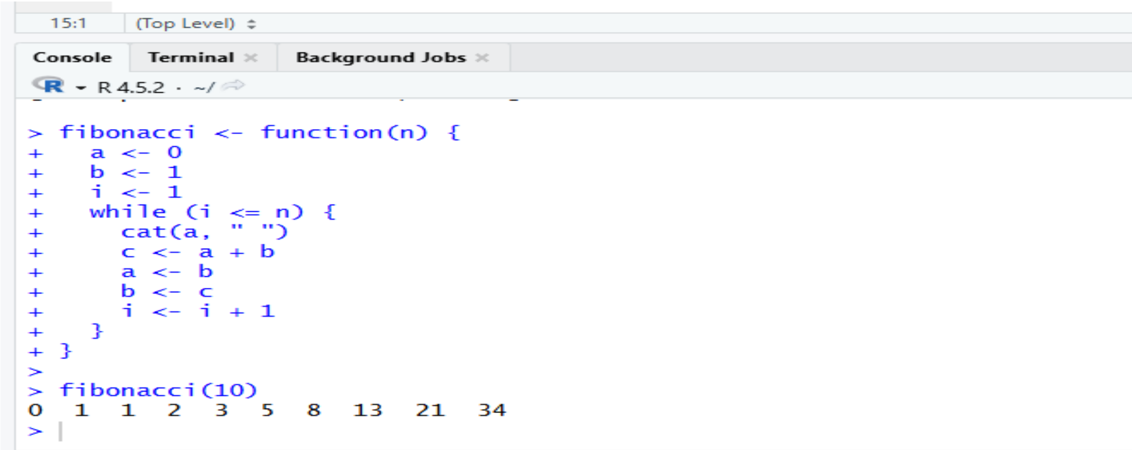
2. Generate the multiplication table of a given number using a for loop.

3. Write a program in R to make a simple calculator.

**Solution 1:**

```
fibonacci <- function(n) {
  a <- 0
  b <- 1
  i <- 1
  while (i <= n) {
    cat(a, " ")
    c <- a + b
    a <- b
    b <- c
    i <- i + 1
  }
}
fibonacci(10)
```

**Output:**

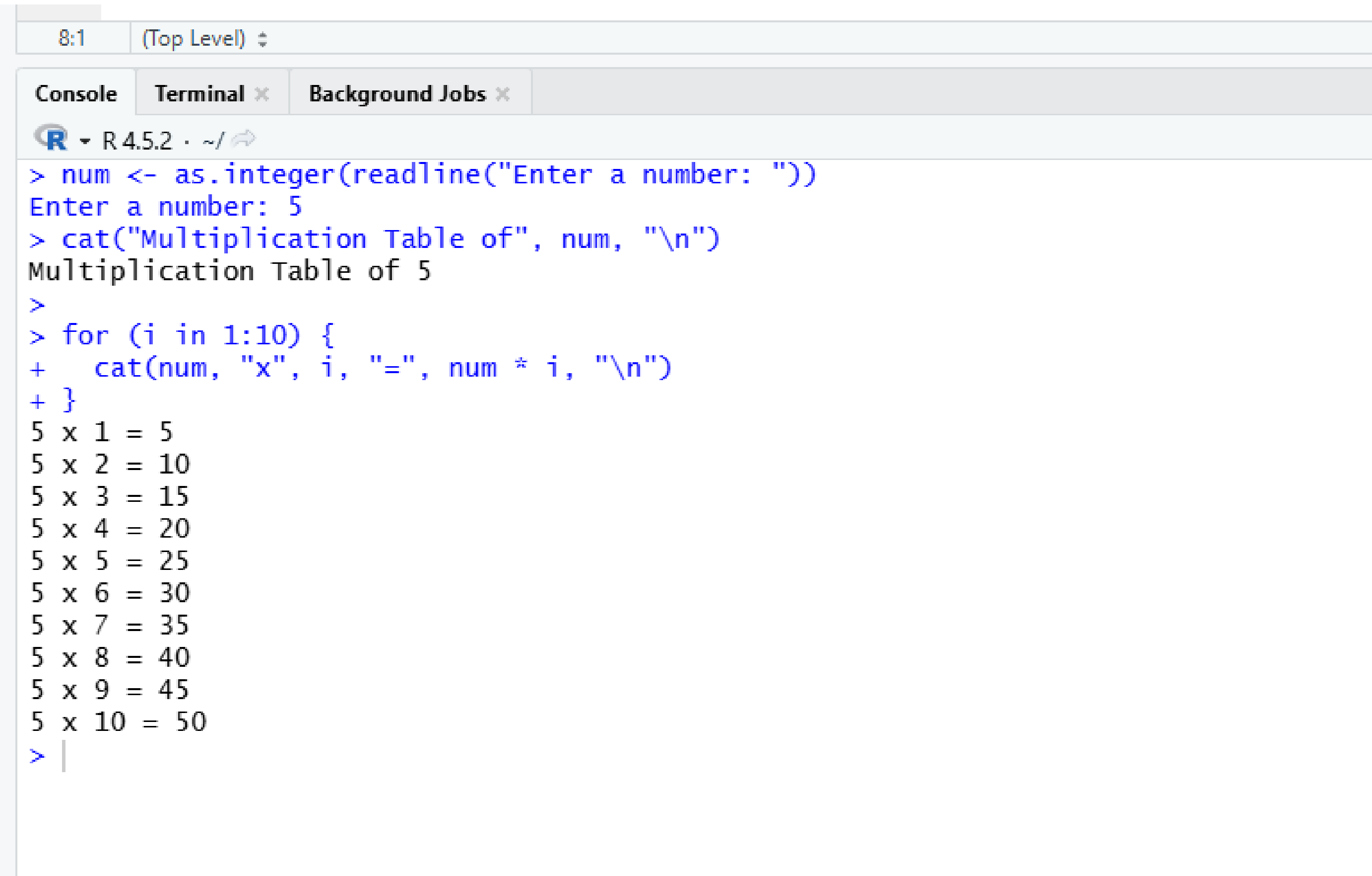


15:1
(Top Level)
Console
Terminal
Background Jobs
R 4.5.2 · ~/
> fibonacci <- function(n) {
+   a <- 0
+   b <- 1
+   i <- 1
+   while (i <= n) {
+     cat(a, " ")
+     c <- a + b
+     a <- b
+     b <- c
+     i <- i + 1
+   }
+ }
>
> fibonacci(10)
0  1  1  2  3  5  8  13  21  34
>

**Solution:2**

```
num <- as.integer(readline("Enter a number: "))

cat("Multiplication Table of", num, "\n")

for (i in 1:10) {

  cat(num, "x", i, "=", num * i, "\n")

}
```

**Output:**



**Solution3:**

```
num1 <- as.numeric(readline(prompt = "Enter the first number: "))
num2 <- as.numeric(readline(prompt = "Enter the second number: "))
op <- readline(prompt = "Enter an operation (+, -, *, /): ")
if (op == "+") {
  result <- num1 + num2
} else if (op == "-") {
  result <- num1 - num2
} else if (op == "*") {
  result <- num1 * num2
} else if (op == "/") {
  if (num2 == 0) {
    result <- "Error: Division by zero is not allowed"
  } else {
    result <- num1 / num2
  }
} else {  result <- "Error: Invalid operation"}
cat("Result:", result, "\n")
```

**Output:**

```
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
> num1 <- as.numeric(readline(prompt = "Enter the first number: "))
Enter the first number: 45
> num2 <- as.numeric(readline(prompt = "Enter the second number: "))
Enter the second number: 25
> op <- readline(prompt = "Enter an operation (+, -, *, /): ")
Enter an operation (+, -, *, /): +
>
> if (op == "+") {
+   result <- num1 + num2
+ } else if (op == "-") {
+   result <- num1 - num2
+ } else if (op == "*") {
+   result <- num1 * num2
+ } else if (op == "/") {
+   if (num2 == 0) {
+     result <- "Error: Division by zero is not allowed"
+   } else {
+     result <- num1 / num2
+   }
+ } else {
+   result <- "Error: Invalid operation"
+ }
>
> # Print the result
> cat("Result:", result, "\n")
Result: 70
> |
```

**Aim: Scoping Rules in R**

- Demonstrate local and global variables using functions.

**Experiment:** Write an R program to demonstrate the difference between local and global variables by defining a variable outside a function and modifying it inside the function. Display the values of the variable before and after function execution.

**Solution:**

```
x <- 10

cat("value of x before function call:", x,"\n")

modify_value <- function(){

  x<- 20

  cat("value of x inside function:", x,"\n")

}

modify_value()

cat("value of x after function call:", x,"\n")
```

**Output:**

```
Console   Terminal ×   Jobs ×
R 3.6.2 · C:/Users/MCA/Desktop/Mr HTML/
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> x <- 10
>
> cat("value of x before function call:", x,"\n")
value of x before function call: 10
>
> modify_value <- function(){
+   x<- 20
+   cat("value of x inside function:", x,"\n")
+ }
>
> modify_value()
value of x inside function: 20
>
> cat("value of x after function call:", x,"\n")
value of x after function call: 10
> 
```

**Aim: Apply Family Functions**

**-** Use apply(), lapply(), sapply(), and tapply().

**Experiment:** Use apply(), lapply(), and sapply() on a matrix and compare the results.

**Solution:**

```
m <- matrix(1:9, nrow = 3)
cat("Matrix:\n")
print(m)
cat("\nUsing apply() - column sums:\n")
print(apply(m, 2, sum))
cat("\nUsing lapply() - column sums:\n")
print(lapply(as.data.frame(m), sum))
cat("\nUsing sapply() - column sums:\n")
print(sapply(as.data.frame(m), sum))
```

## Output:

```
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
> m <- matrix(1:9, nrow = 3)
>
> cat("Matrix:\n")
Matrix:
> print(m)
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
>
> cat("\nUsing apply() - column sums:\n")

Using apply() - column sums:
> print(apply(m, 2, sum))
[1]  6 15 24
>
> cat("\nUsing lapply() - column sums:\n")

Using lapply() - column sums:
> print(lapply(as.data.frame(m), sum))
$V1
[1] 6

$V2
[1] 15

$V3
[1] 24

>
> cat("\nUsing sapply() - column sums:\n")

Using sapply() - column sums:
> print(sapply(as.data.frame(m), sum))
V1 V2 V3
 6 15 24
> |
```

**Aim: Working with Date, Time, and Sleep Functions**

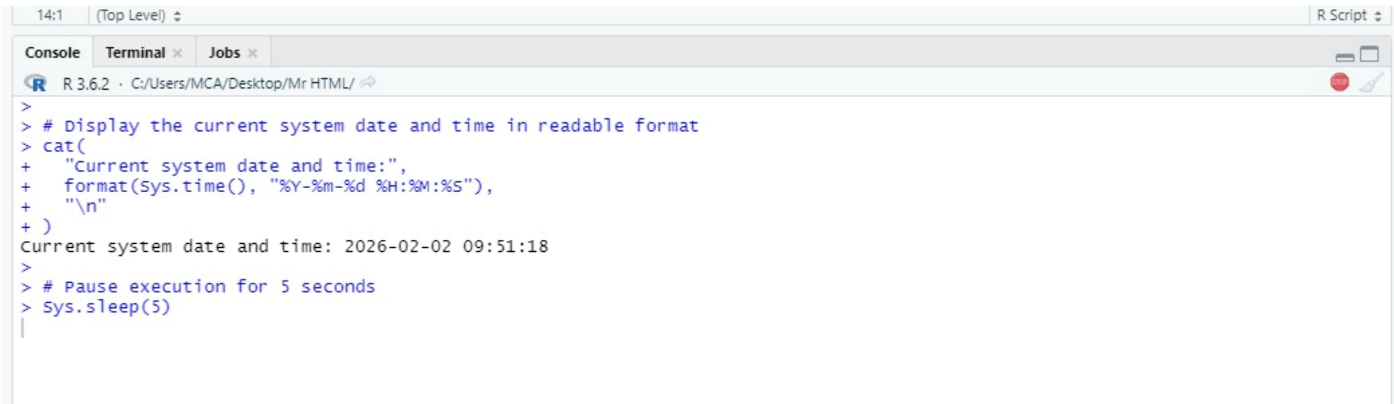
Handle system time, dates, and implement delays using Sys.sleep().

**Experiment:** Write a program to display the current system date and time and pause execution for 5 seconds.

**Solution:**

```
 # Display the current system date and time in readable format
cat(
  "Current system date and time:",
  format(Sys.time(), "%Y-%m-%d %H:%M:%S"),
  "\n"
)
# Pause execution for 5 seconds
Sys.sleep(5)
# Message after pause
cat("Program resumed after 5 seconds.\n")
```

**Output:**



**Aim: Reading and Writing Data Files**

- Read/write CSV and text files using read.csv() and write.csv().

**Experiment:** Read a CSV file into R and display summary statistics of the data.

**Solution:**

```
data <- read.csv("C:/Users/MCA/Desktop/r prog/data.csv",
                 header = TRUE,
                 stringsAsFactors = FALSE)
head(data)
str(data)
summary(data)
numeric_data <- data[sapply(data, is.numeric)]
apply(numeric_data, 2, function(x) {
  c(
    Min    = min(x, na.rm = TRUE),
    Max    = max(x, na.rm = TRUE),
    Mean   = mean(x, na.rm = TRUE),
    Median = median(x, na.rm = TRUE),
    SD     = sd(x, na.rm = TRUE)
  )
})
```

**Output:**

```
Console   Terminal ×   Jobs ×
R 3.6.2 · C:/Users/MCA/Desktop/Mr HTML/
>
> apply(numeric_data, 2, function(x) {
+   c(
+     Min    = min(x, na.rm = TRUE),
+     Max    = max(x, na.rm = TRUE),
+     Mean   = mean(x, na.rm = TRUE),
+     Median = median(x, na.rm = TRUE),
+     SD     = sd(x, na.rm = TRUE)
+   )
+ })
           X345    X4356
Min     3567.00  647.000
Max    54789.00 6474.000
Mean   37678.00 4198.333
Median 54678.00 5474.000
SD     29541.04 3115.923
> data <- read.csv("C:/Users/MCA/Desktop/r prog/data.c
+                  header = TRUE,
+                  stringsAsFactors = FALSE)
>
> head(data)
   X345 X4356
1  3567   647
2 54789  6474
3 54678  5474
>
> str(data)
'data.frame':	3 obs. of  2 variables:
 $ X345 : int  3567 54789 54678
 $ X4356: int  647 6474 5474
>
> summary(data)
      X345           X4356     
 Min.   : 3567   Min.   : 647  
 1st Qu.:29123   1st Qu.:3060  
 Median :54678   Median :5474  
 Mean   :37678   Mean   :4198  
 3rd Qu.:54734   3rd Qu.:5974  
 Max.   :54789   Max.   :6474  
>
> numeric_data <- data[sapply(data, is.numeric)]
>
> apply(numeric_data, 2, function(x) {
+   c(
+     Min    = min(x, na.rm = TRUE),
+     Max    = max(x, na.rm = TRUE),
+     Mean   = mean(x, na.rm = TRUE),
+     Median = median(x, na.rm = TRUE),
+     SD     = sd(x, na.rm = TRUE)
+   )
+ })
           X345    X4356
Min     3567.00  647.000
Max    54789.00 6474.000
Mean   37678.00 4198.333
Median 54678.00 5474.000
SD     29541.04 3115.923
> |
```

**Aim: Saving and Loading R Data Objects**

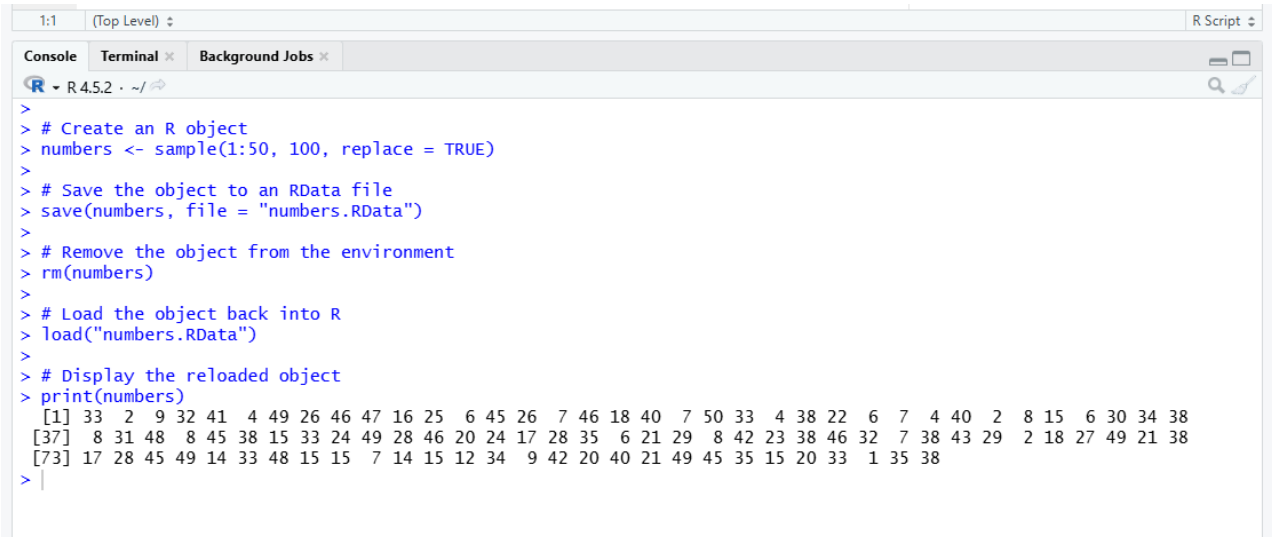
Use save(), load(), and RData files.

**Experiment:** Create an R object, save it to a file, and reload it.

**Solution:**

```
# Create an R object
numbers <- sample(1:50, 100, replace = TRUE)
# Save the object to an RData file
save(numbers, file = "numbers.RData")
# Remove the object from the environment
rm(numbers)
# Load the object back into R
load("numbers.RData")
# Display the reloaded object
print(numbers)
```

**Output:**




```
>
> # Create an R object
> numbers <- sample(1:50, 100, replace = TRUE)
>
> # Save the object to an RData file
> save(numbers, file = "numbers.RData")
>
> # Remove the object from the environment
> rm(numbers)
>
> # Load the object back into R
> load("numbers.RData")
>
> # Display the reloaded object
> print(numbers)
  [1] 33  2  9 32 41  4 49 26 46 47 16 25  6 45 26  7 46 18 40  7 50 33  4 38 22  6  7  4 40  2  8 15  6 30 34 38
 [37]  8 31 48  8 45 38 15 33 24 49 28 46 20 24 17 28 35  6 21 29  8 42 23 38 46 32  7 38 43 29  2 18 27 49 21 38
 [73] 17 28 45 49 14 33 48 15 15  7 14 15 12 34  9 42 20 40 21 49 45 35 15 20 33  1 35 38
>
```

**Aim: Generating Data in R**

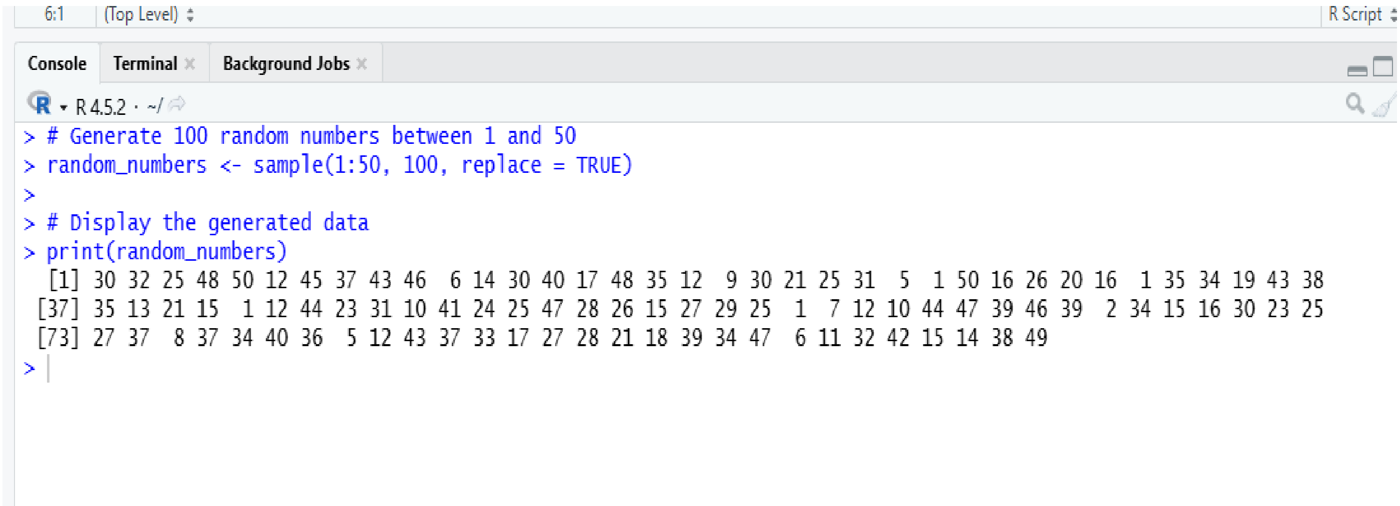
**-** Generate sequences and random data using built-in functions.

**Experiment:** Generate 100 random numbers between 1 and 50 and store them in a vector.

**Solution:**

```
# Generate 100 random numbers between 1 and 50

random_numbers <- sample(1:50, 100, replace = TRUE)

# Display the generated data

print(random_numbers)
```

**Output:**



```
6:1    (Top Level)                                                    R Script
Console   Terminal ×   Background Jobs ×
R 4.5.2 · ~/
> # Generate 100 random numbers between 1 and 50
> random_numbers <- sample(1:50, 100, replace = TRUE)
>
> # Display the generated data
> print(random_numbers)
  [1] 30 32 25 48 50 12 45 37 43 46  6 14 30 40 17 48 35 12  9 30 21 25 31  5  1 50 16 26 20 16  1 35 34 19 43 38
 [37] 35 13 21 15  1 12 44 23 31 10 41 24 25 47 28 26 15 27 29 25  1  7 12 10 44 47 39 46 39  2 34 15 16 30 23 25
 [73] 27 37  8 37 34 40 36  5 12 43 37 33 17 27 28 21 18 39 34 47  6 11 32 42 15 14 38 49
>
```

**Aim: Regular and Random Sequences**

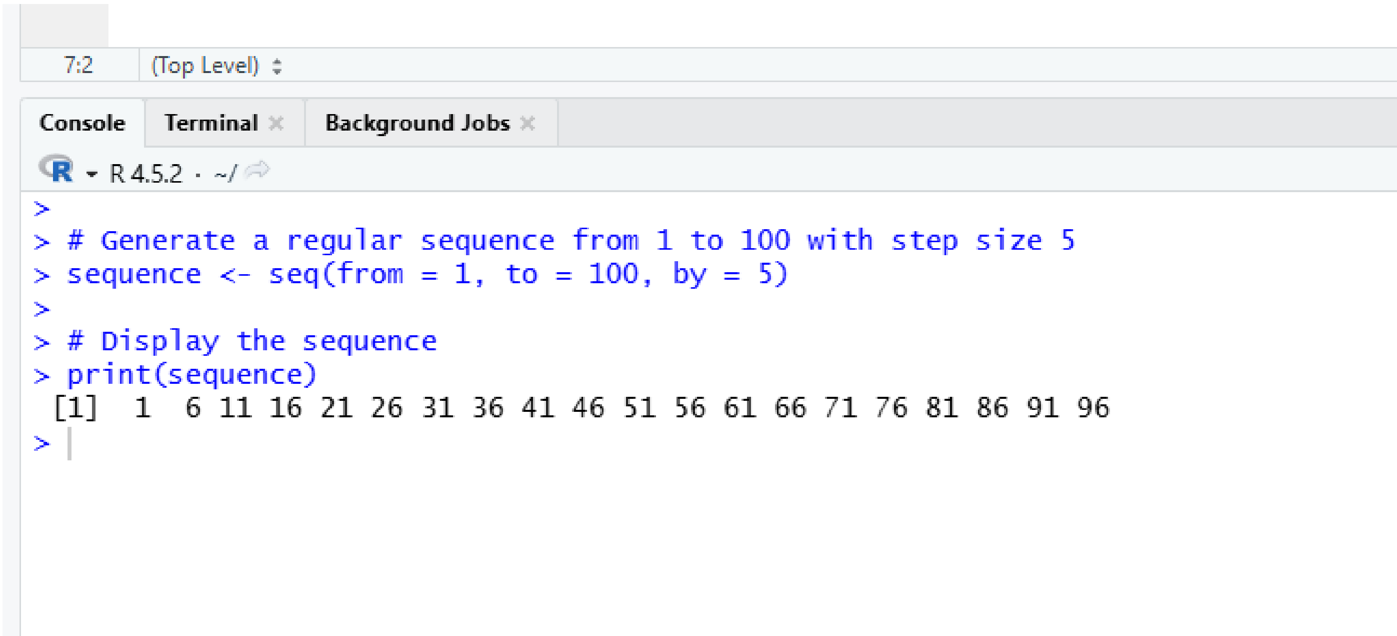
Use seq(), rep(), runif(), rnorm().

**Experiment:** Generate a regular sequence from 1 to 100 with a step size of 5.

**Solution :**

```
# Generate a regular sequence from 1 to 100 with step size 5

sequence <- seq(from = 1, to = 100, by = 5)

# Display the sequence

print(sequence)
```

**Output:**



**Aim: Handling Missing Data**

- Use is.na(), na.omit(), and complete.cases().

**Experiment:** Write a program to identify and replace missing values with the mean.

**Solution :**

```
# data <- read.csv("C:/Users/MCA/Desktop/r prog/data.csv",
                 header = TRUE,
                 stringsAsFactors = FALSE)
colSums(is.na(data))
for (i in names(data)) {
  if (is.numeric(data[[i]])) {
    data[[i]][is.na(data[[i]])] <- mean(data[[i]], na.rm = TRUE)
  }
}
colSums(is.na(data))
summary(data)
```

**Output:**

```
Console   Terminal ×   Jobs ×
R 3.6.2 · C:/Users/MCA/Desktop/Mr HTML/
> colSums(is.na(data))
 X345 X4356
    0     0
> for (i in names(data)) {
+   if (is.numeric(data[[i]])) {
+     data[[i]][is.na(data[[i]])] <- mean(data[[i]], na.rm = TRUE)
+   }
+ }
> colSums(is.na(data))
 X345 X4356
    0     0
> 
```

**Aim: Creating and Manipulating Objects**

Work with vectors, matrices, lists, and data frames.

**Experiment:** Write a R program to create a data frame and get the structure, statistical summary and nature of the data of a given data frame.

**Solution :**

```
data <- data.frame(
  ID    = c(1, 2, 3, 4, 5),
  Name  = c("Amit", "Riya", "Karan", "Neha", "Vijay"),
  Age   = c(21, 22, 20, 23, 21),
  Marks = c(78, 85, 69, 90, 74),
  Grade = factor(c("B", "A", "C", "A", "B"))
)
print(data)
cat("\nStructure of the Data Frame:\n")
str(data)
cat("\nStatistical Summary:\n")
summary(data)
cat("\nNature of the Data:\n")
sapply(data, class)
```

**Output:**

```
Go to file/function    Addins
Source
Console   Terminal   Jobs
R 3.6.2 · C:/Users/MCA/Desktop/Mr HTML/
> colSums(is.na(data))
 X345 X4356
    0     0
> for (i in names(data)) {
+   if (is.numeric(data[[i]])) {
+     data[[i]][is.na(data[[i]])] <- mean(data[[i]], na.rm = TRUE)
+   }
+ }
> colSums(is.na(data))
 X345 X4356
    0     0
> data <- data.frame(
+   ID    = c(1, 2, 3, 4, 5),
+   Name  = c("Amit", "Riya", "Karan", "Neha", "Vijay"),
+   Age   = c(21, 22, 20, 23, 21),
+   Marks = c(78, 85, 69, 90, 74),
+   Grade = factor(c("B", "A", "C", "A", "B"))
+ )
>
> print(data)
  ID  Name Age Marks Grade
1  1  Amit  21    78     B
2  2  Riya  22    85     A
3  3 Karan  20    69     C
4  4  Neha  23    90     A
5  5 Vijay  21    74     B
>
> cat("\nStructure of the Data Frame:\n")

Structure of the Data Frame:
> str(data)
'data.frame':	5 obs. of  5 variables:
 $ ID   : num  1 2 3 4 5
 $ Name : Factor w/ 5 levels "Amit","Karan",..: 1 4 2 3 5
 $ Age  : num  21 22 20 23 21
 $ Marks: num  78 85 69 90 74
 $ Grade: Factor w/ 3 levels "A","B","C": 2 1 3 1 2
>
> cat("\nStatistical Summary:\n")

Statistical Summary:
> summary(data)
       ID         Name        Age            Marks        Grade
 Min.   :1   Amit :1   Min.   :20.0   Min.   :69.0   A:2
 1st Qu.:2   Karan:1   1st Qu.:21.0   1st Qu.:74.0   B:2
 Median :3   Neha :1   Median :21.0   Median :78.0   C:1
 Mean   :3   Riya :1   Mean   :21.4   Mean   :79.2
 3rd Qu.:4   Vijay:1   3rd Qu.:22.0   3rd Qu.:85.0
 Max.   :5             Max.   :23.0   Max.   :90.0
>
> cat("\nNature of the Data:\n")

Nature of the Data:
> sapply(data, class)
       ID      Name       Age     Marks     Grade
"numeric"  "factor" "numeric" "numeric"  "factor"
>
```

**Aim: Indexing and Accessing Object Values**

Demonstrate indexing systems for different R objects.

**Experiment:** Write a R program to create a matrix and perform following operations:

i. Extract the sub matrix whose rows have column value from a given matrix.

ii. Convert a matrix to a 1 dimensional array.

**Solution 1 :**

```
# R Program: Create a matrix and extract submatrix based on column value
# Step 1: Create a matrix
mat <- matrix(
  c(10, 20, 30,
    40, 50, 60,
    70, 80, 90,
    100, 110, 120),
  nrow = 4,
  byrow = TRUE
)
cat("Original Matrix:\n")
print(mat)
# Step 2: Extract submatrix where column 2 value > 50
sub_mat <- mat[mat[, 2] > 50, ]
cat("\nSubmatrix (Rows with Column 2 > 50):\n")
print(sub_mat)
)
```

**Output:**

```
1:2    (Top Level) ÷

Console   Terminal ×   Background Jobs ×

R 4.5.2 · ~/
> # R Program: Create a matrix and extract submatrix based on column value
>
> # Step 1: Create a matrix
> mat <- matrix(
+   c(10, 20, 30,
+     40, 50, 60,
+     70, 80, 90,
+     100, 110, 120),
+   nrow = 4,
+   byrow = TRUE
+ )
>
> cat("Original Matrix:\n")
Original Matrix:
> print(mat)
     [,1] [,2] [,3]
[1,]   10   20   30
[2,]   40   50   60
[3,]   70   80   90
[4,]  100  110  120
>
> # Step 2: Extract submatrix where column 2 value > 50
> sub_mat <- mat[mat[, 2] > 50, ]
>
> cat("\nSubmatrix (Rows with Column 2 > 50):\n")

Submatrix (Rows with Column 2 > 50):
> print(sub_mat)
     [,1] [,2] [,3]
[1,]   70   80   90
[2,]  100  110  120
> 
```

**Solution 2 :**

```
# R Program: Convert a matrix to a 1-dimensional array
# Step 1: Create a matrix
mat <- matrix(
  c(10, 20, 30,
    40, 50, 60,
    70, 80, 90,
    100, 110, 120),
  nrow = 4,
  byrow = TRUE
)
cat("Original Matrix:\n")
print(mat)
# Step 2: Convert matrix to 1-dimensional array
one_d_array <- as.vector(mat)
cat("\n1-Dimensional Array:\n")
print(one_d_array)
```

**Output:**

21:1 (Top Level)

Console | Terminal | Background Jobs

R 4.5.2 · ~/

```
>
> # R Program: Convert a matrix to a 1-dimensional array
>
> # Step 1: Create a matrix
> mat <- matrix(
+   c(10, 20, 30,
+     40, 50, 60,
+     70, 80, 90,
+     100, 110, 120),
+   nrow = 4,
+   byrow = TRUE
+ )
>
> cat("Original Matrix:\n")
Original Matrix:
> print(mat)
     [,1] [,2] [,3]
[1,]   10   20   30
[2,]   40   50   60
[3,]   70   80   90
[4,]  100  110  120
>
> # Step 2: Convert matrix to 1-dimensional array
> one_d_array <- as.vector(mat)
>
> cat("\n1-Dimensional Array:\n")

1-Dimensional Array:
> print(one_d_array)
 [1]  10  40  70 100  20  50  80 110  30  60  90 120
>
```

**Aim: Introduction to S3 and S4 Classes**

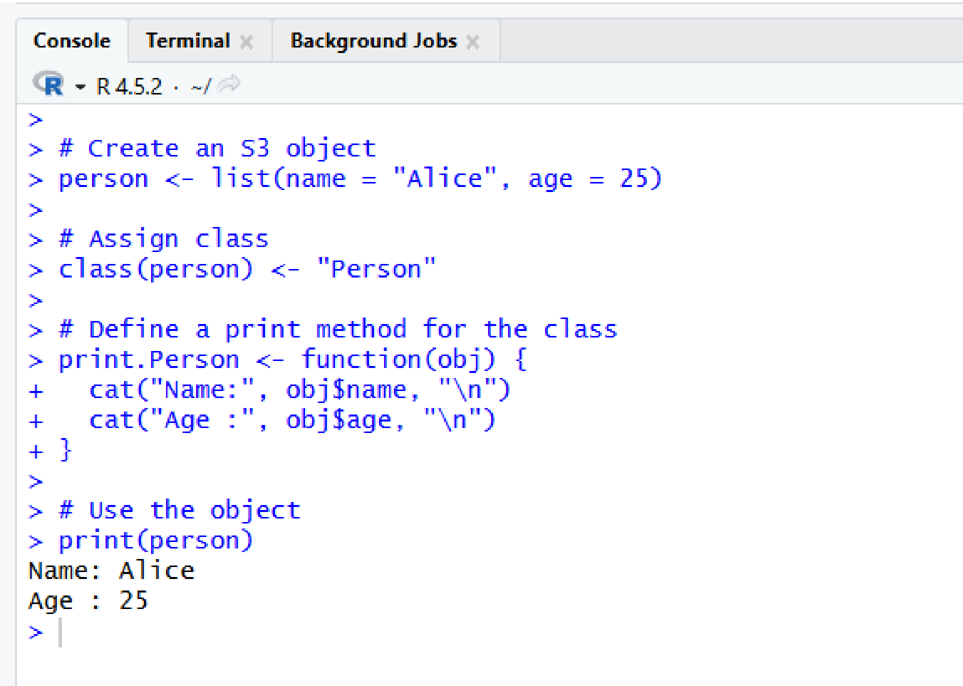
Create and use simple S3 and S4 objects.

**Experiment:** Demonstrate indexing on vectors, matrices, and data frames.

**Solution :**

```
# Create an S3 object
person <- list(name = "Alice", age = 25)
# Assign class
class(person) <- "Person"
# Define a print method for the class
print.Person <- function(obj) {
  cat("Name:", obj$name, "\n")
  cat("Age :", obj$age, "\n")
}
# Use the object
print(person)
```

**Output:**




```
>
> # Create an S3 object
> person <- list(name = "Alice", age = 25)
>
> # Assign class
> class(person) <- "Person"
>
> # Define a print method for the class
> print.Person <- function(obj) {
+   cat("Name:", obj$name, "\n")
+   cat("Age :", obj$age, "\n")
+ }
>
> # Use the object
> print(person)
Name: Alice
Age : 25
>
```

**Aim: Reference Classes and User-Defined Functions**

**Implement reference classes and arithmetic functions.**

**Experiment:** Create a reference class to perform basic arithmetic operations.

**Solution :**

```
MyClass <- setRefClass("MyClass",
                fields = list(x = "numeric", y = "numeric"),
                methods = list(
                  showValues = function() {
                    cat("x =", x, ", y =", y, "\n")
                  }
                ))
obj <- MyClass$new(x = 10, y = 20)
# Access fields
obj$x
obj$y
obj$showValues()
```

**Output:**

```
Console    Terminal    Background Jobs
R 4.5.2 · ~/
> MyClass <- setRefClass("MyClass",
+                        fields = list(x = "numeric", y = "numeric"),
+                        methods = list(
+                          showValues = function() {
+                            cat("x =", x, ", y =", y, "\n")
+                          }
+                        ))
> obj <- MyClass$new(x = 10, y = 20)
> # Access fields
> obj$x
[1] 10
> obj$y
[1] 20
> obj$showValues()
x = 10 , y = 20
> 
```

**Aim: Creating Basic 2D and 3D Plots**

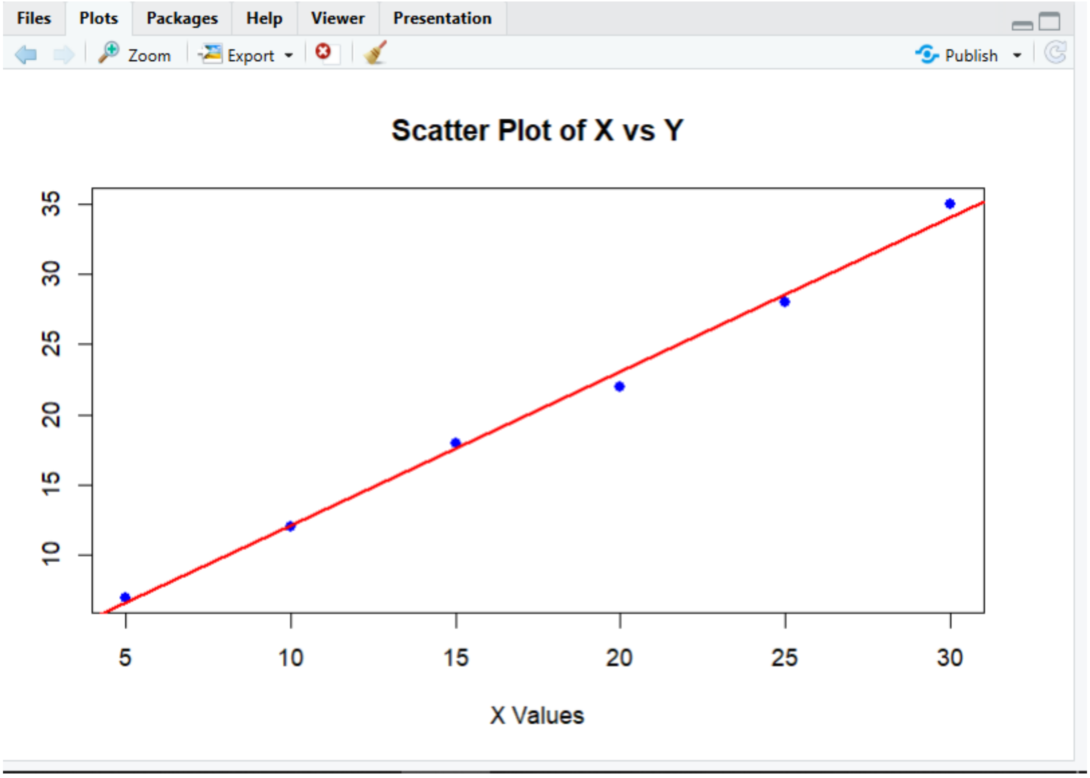
- Plot line graphs, scatter plots, and bar charts.

**Experiment:** Create a scatter plot for two numeric variables.

**Solution :**

```
# Create numeric variables
x <- c(5, 10, 15, 20, 25, 30)
y <- c(7, 12, 18, 22, 28, 35)
# Create scatter plot
plot(x, y,
     main = "Scatter Plot of X vs Y",
     xlab = "X Values",
     ylab = "Y Values",
     col = "blue",
     pch = 19)
# Add regression line
abline(lm(y ~ x), col = "red", lwd = 2)
```

**Output:**


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Scatter Plot of X vs Y
10
15
20
25
30
35
5
10
15
20
25
30
X Values

**Aim: Customizing Graphs in R**

Add titles, labels, legends, colors, and themes.

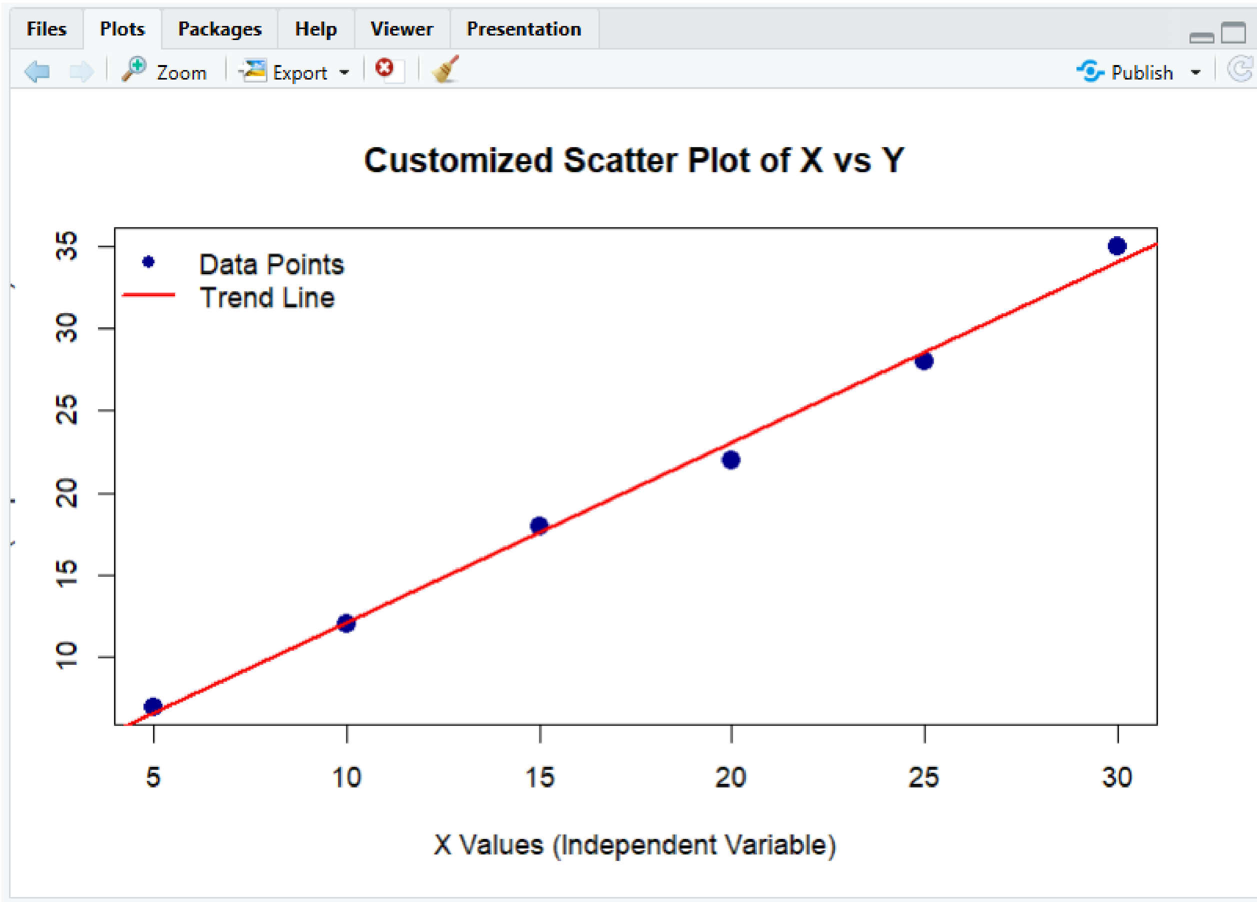
**Experiment:** Customize the plot created in exp16 by adding title, axis labels, and legend.

**Solution :**

```
x <- c(5, 10, 15, 20, 25, 30)

y <- c(7, 12, 18, 22, 28, 35)

# Custom scatter plot

plot(x, y,

     main = "Customized Scatter Plot of X vs Y",

     xlab = "X Values (Independent Variable)",

     ylab = "Y Values (Dependent Variable)",

     col = "darkblue",

     pch = 19,

     cex = 1.5)

# Add regression line

abline(lm(y ~ x), col = "red", lwd = 2)

# Add legend

legend("topleft",

       legend = c("Data Points", "Trend Line"),

       col = c("darkblue", "red"),

       pch = c(19, NA),
```

```
lty = c(NA, 1),
    lwd = c(NA, 2),
    bty = "n”
```

**Output:**



**Aim: One-Way ANOVA Using R**

Perform and interpret one-way ANOVA.

**Experiment:** Perform one-way ANOVA on a dataset and interpret the result.

**Solution :**

```
# Step 1: Create sample data
# Suppose we have test scores of students in 3 different teaching methods
scores <- c(85, 90, 78, 92, 88, 76, 95, 89, 84, 91, 87, 80)
method <- factor(c(rep("Method1", 4),
              rep("Method2", 4),
              rep("Method3", 4)))
# Step 2: Combine into a data frame
data <- data.frame(scores, method)
# Step 3: Perform one-way ANOVA
anova_result <- aov(scores ~ method, data = data)
# Step 4: View ANOVA table
summary(anova_result)
```

**Output:**

```
Console   Terminal   Background Jobs
R 4.5.2 · ~/
> # Step 1: Create sample data
> # Suppose we have test scores of students in 3 different teaching methods
> scores <- c(85, 90, 78, 92, 88, 76, 95, 89, 84, 91, 87, 80)
> method <- factor(c(rep("Method1", 4),
+                   rep("Method2", 4),
+                   rep("Method3", 4)))
>
> # Step 2: Combine into a data frame
> data <- data.frame(scores, method)
>
> # Step 3: Perform one-way ANOVA
> anova_result <- aov(scores ~ method, data = data)
>
> # Step 4: View ANOVA table
> summary(anova_result)
            Df Sum Sq Mean Sq F value Pr(>F)
method       2    4.5    2.25   0.054  0.947
Residuals    9  371.8   41.31
> 
```

**Aim: Linear Regression Analysis**

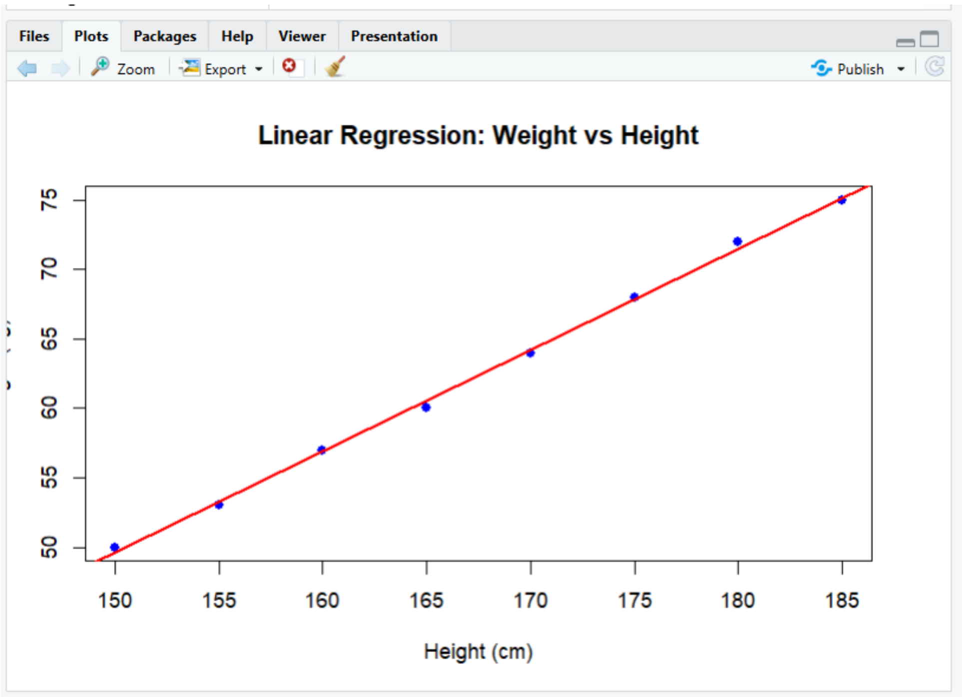
Implement linear regression, interpret model output, and visualize results.

**Experiment:** Fit a linear regression model and plot the regression line.

**Solution :**

```
height <- c(150, 155, 160, 165, 170, 175, 180, 185)

weight <- c(50, 53, 57, 60, 64, 68, 72, 75)

# Step 2: Fit linear regression model

model <- lm(weight ~ height)

# Step 3: View model summary

summary(model)

# Step 4: Plot scatter plot with regression line

plot(height, weight,

    main = "Linear Regression: Weight vs Height",

    xlab = "Height (cm)",

    ylab = "Weight (kg)",

    pch = 19, col = "blue")

abline(model, col = "red", lwd = 2)
```

**Output:**


Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Export
Publish
Linear Regression: Weight vs Height
50
55
60
65
70
75
150
155
160
165
170
175
180
185
Height (cm)